

Technical Assessment

Design, Plan & Ship with AI

Role: Full Stack Developer (TypeScript)

Estimated Time: 4-8 Hours

Thank you for your interest in joining our engineering team. This technical task is designed to evaluate how you structure code, design APIs, and integrate web and mobile applications. The focus of this task is clean architecture, code organization, and maintainability, rather than building a complex UI.

What this measures: How you think through a problem, design a system, and use AI tools to execute fast; not how many features you can hand-code. **This is not a coding marathon.** We have deliberately shifted the focus away from grinding out a full implementation. We want to see how you *own a problem end to end*: scope it, design the architecture, make trade-off decisions, and direct AI tools to build a working slice quickly. A senior engineer today is judged as much by *how they leverage AI* as by raw coding ability.

Overview

You will design and partially build a Wellness Package Management System. The product itself is intentionally simple so that your design thinking and AI workflow (not the domain) are what stand out.

The system has three surfaces:

1. Admin Portal (Next.js / TypeScript): administrators manage wellness packages.
2. Backend API (NestJS / TypeScript): serves both the admin portal and the mobile app.

3. Mobile App (React Native / TypeScript): end users browse available packages.

Core domain entity, Wellness Package:

- id, name, description, price, duration_minutes, created_at

You are free and encouraged to extend this model where it strengthens your design (e.g. categories, availability, status, audit fields). Justify any additions in your plan.

What We Are Really Evaluating

We care about three things, weighted equally:

Dimension	What strong looks like
1. Planning & Design	Clear scoping, a coherent architecture, sensible data model, well-defined API contracts, and explicit trade-offs. You can explain why, not just what.
2. AI-Assisted Execution	You direct AI tools (Claude, Cursor, Copilot, etc.) deliberately good prompts, good decomposition, reviewing and correcting AI output rather than pasting it blindly. You move fast without losing control of quality.
3. Ownership of Full Stack	You can speak credibly to all three repos (backend, web, mobile), how they fit together, and how you'd take this from prototype to production solo.

*We are NOT evaluating: visual polish, feature count, or whether every endpoint is fully implemented. A thin, well-reasoned slice beats a sprawling, unexplained one.

Part A: Design & Plan

This is the heart of the assessment, producing a concise design document (Markdown, PDF, Notion, or slides; your choice) covering the following. This aims for clarity and decisiveness over length.

1. Problem framing & scope

- What you chose to build in the time available, and just as important what you deliberately left out.
- Any assumptions you made about users, scale, or requirements.

2. Architecture

- A high-level diagram showing how the three surfaces and the database relate.
- How code is organized across the backend, admin portal, and mobile app (modules, layers, separation of concerns).
- Where shared concerns live (types, validation, error handling, config).

3. Data model & API contract

- Your schema for the package entity (and any extensions), with reasoning.
- The full API surface: routes, methods, request/response shapes, status codes, validation rules. Note where admin and mobile APIs differ and why.

4. Key technical decisions & trade-offs

- 3–5 decisions you made and the alternatives you rejected (e.g. database choice, state management, how the mobile app talks to the API, auth approach if any).
- What you would do differently with more time or at production scale.

5. Your AI workflow

This section is required and is a major signal for us.

- Which AI tools you used and for what (scaffolding, boilerplate, debugging, design review, tests).
- 2–3 prompts or prompt sequences you're proud of and why they worked.
- At least one example where the AI got it wrong, how you caught it, and how you corrected the course.
- Where you chose NOT to use AI, and why.

Part B: Thin Prototype (proof you can execute)

Build a vertical slice that proves your design works end to end. We want depth on one path, not breadth across many. Use AI freely to move fast.

Minimum bar, one feature working across the whole stack:

- Backend: at least GET /mobile/packages and the admin CRUD for packages (NestJS + TypeScript and DB: MySQL).
- Admin Portal: a working list + create/edit/delete against the real API.
- Mobile App: a Flutter screen that fetches and displays packages from the same API.

It is acceptable to stub, fake, or omit secondary features, as long as you can explain in the session how you'd complete them. A polished prototype of the wrong thing scores worse than a rough prototype of the right thing, clearly explained.

Bonus

- Docker / docker-compose for one-command spin-up
- Swagger / OpenAPI docs
- A handful of unit tests on the service layer
- Environment configuration / .env handling

The Live Session (1 hour)

Segment	Time	What to cover
Walkthrough	~20 min	Walk us through how you used AI, show real prompts, a moment the AI was wrong, and how you stayed in control of quality. Present your design, the problem framing, architecture, data/API design.
Q&A / live	~30 min	A general Question & Answer session.

What to Submit (before the session)

1. Public GitHub repo with /backend, /admin-portal, /mobile-app.
2. Design document (Part A) in the repo (e.g. /docs) or linked.
3. README with project structure, setup steps, and how to run each surface.
4. AI workflow notes can live inside the design doc.
5. A few screenshots: admin page, mobile screen, a sample API response.